

Materi

Mata Kuliah

Pemrograman Berorientasi
Objek



Nama : _____

Kelas
PTI A 2022

DAFTAR ISI

DAFTAR ISI	i
DAFTAR TABEL	ii
BAB 1 CLASS dan OBJECT	1
1.1 Pengantar Pemrograman.....	1
1.2 Anatomi Class.....	1
1.3 Deklarasi dan Instansiasi Objek.....	3
1.4 Mengakses Anggota Class	4
1.5 Interaksi Antar Objek	6
BAB 2 ENKAPSULASI	9
2.1 Konsep Data Hiding.....	9
2.2 Access Modifier.....	12
2.3 Method Getter dan Setter	14
BAB 3 INHERITANCE (PEWARISAN)	17
3.1 Konsep Pewarisan Sifat.....	17
3.2 Pemanggilan Konstruktor Induk	19
3.3 Hierarki Pewarisan Bertingkat	21
BAB 4 POLIMORFISME	25
4.1 Pengantar Polimorfisme	25
4.2 Upcasting dan Downcasting.....	26
4.3 Polimorfisme pada Banyak Objek	29
DAFTAR PUSTAKA.....	32

DAFTAR TABEL

Tabel 1. 1 Contoh Penerapan Kode Anatomi Class	1
Tabel 1. 2 Contoh Penerapan Deklarasi dan Instansiasi	3
Tabel 1. 3 Contoh Penerapan Kode Akses Anggota Kelas	5
Tabel 1. 4 Contoh Penerapan Interaksi Antar Objek	6
Tabel 2. 1 Contoh Penerapan Data Hiding	10
Tabel 2. 2 Contoh Penerapan Access Modifier	12
Tabel 2. 3 Contoh Penerapan Method Getter dan Setter	15
Tabel 3. 1 Contoh Penerapan Inheritance	17
Tabel 3. 2 Contoh Penerapan Pemanggilan Konstruktor Induk	20
Tabel 3. 3 Contoh Penerapan Pewarisan Bertingkat	22
Tabel 4. 1 Contoh Penerapan Polimorfisme	25
Tabel 4. 2 Contoh Penerapan Upcasting dan Downcasting	27
Tabel 4. 3 Contoh Penerapan Polomorfisme pada Banyak Objek	30

BAB 1

CLASS dan OBJECT

1.1 Pengantar Pemrograman

Pemrograman adalah pendekatan developer dalam menyelesaikan masalah komputasi, yang secara umum terbagi menjadi Pemrograman Prosedural dan Berorientasi Objek (PBO). Pemrograman prosedural (seperti C dan Pascal) mengeksekusi instruksi secara berurutan dengan memisahkan fungsi dan data, di mana data bersifat global sehingga rentan memicu error dan sulit dipelihara pada program berskala besar. Sebaliknya, PBO (seperti Java, C++, dan PHP) memecahkan masalah dengan memodelkannya menjadi objek-objek mandiri yang membungkus rapat data (atribut) dan fungsinya (*Method*) menjadi satu kesatuan. Pendekatan PBO ini membuat sistem jauh lebih aman dari modifikasi sembarangan, fleksibel, mudah dipelihara, dan kodenya dapat digunakan ulang (reusable) untuk proyek lainnya.

1.2 Anatomi Class

Class dalam Pemrograman Berorientasi Objek (PBO) berfungsi sebagai cetakan atau *blueprint*, ibarat formulir pendaftaran yang masih kosong. Wujud nyata dari cetakan tersebut setelah datanya diisi disebut sebagai Objek (*Object*).

Secara teknis, anatomi sebuah *Class* dibangun oleh tiga komponen utama:

- A. **Atribut (Data):** Variabel yang menempel pada *Class* untuk menyimpan informasi atau status dari objek tersebut (contoh: *namaSiswa*, *nisn*, *nilaiPBO*). Setiap atribut wajib memiliki tipe data yang jelas seperti *String* atau *int*.
- B. **Method (Aksi):** Kumpulan blok kode yang menentukan perilaku (*behavior*) atau apa yang bisa dilakukan oleh objek. *Method* digunakan untuk memanipulasi, mengolah, atau menampilkan data atribut, dan di sinilah logika program (seperti *if-else*) dituliskan.
- C. **Constructor:** *Method* khusus yang dieksekusi secara otomatis satu kali pada detik pertama objek diciptakan. Fungsinya adalah untuk mengatur nilai awal data objek tersebut. Ciri mutlak *constructor* adalah namanya wajib sama persis dengan nama *Class* dan tidak memiliki tipe kembalian.

Tabel 1. 1 Contoh Penerapan Kode Anatomi *Class*

```
public Class Buku {  
  
    // 1. Atribut (Data)  
    public String judul;  
    public String pengarang;  
    public int harga;  
  
    // 2. Constructor (Pencetak Awal)
```

```

    public Buku(String inputJudul, String inputPengarang,
int inputHarga) {
        judul = inputJudul;
        pengarang = inputPengarang;
        harga = inputHarga;
    }

    // 3. Method (Aksi)
    public void tampilkanInfo() {
        System.out.println("Judul Buku : " + judul);
        System.out.println("Pengarang : " + pengarang);
    }

    public void beriDiskon(int potongan) {
        harga = harga - potongan;
        System.out.println("Harga setelah diskon: Rp" +
harga);
    }

    // =====
    // 4. MAIN METHOD (Pintu Masuk Program)
    // =====
    public static void main(String[] args) {

        System.out.println("--- PROGRAM DIMULAI ---");

        // Menciptakan objek nyata dari cetakan Buku
        Buku bukuPBO = new Buku("Tutorial Java untuk
Pemula", "Pak Budi", 100000);

        // Memanggil Method
        bukuPBO.tampilkanInfo();
        bukuPBO.beriDiskon(20000);

        System.out.println("--- PROGRAM SELESAI ---");
    }
}

```

Penjelasan Singkat Eksekusi:

1. Saat program dijalankan, baris `new Buku(...)` akan memanggil Constructor. Data "Tutorial Java untuk Pemula", "Pak Budi", dan 100000 akan dimasukkan ke dalam Atribut.
2. Perintah `bukuPBO.tampilkanInfo()` memanggil *Method* pertama untuk mencetak teks ke layar berdasarkan data yang sudah disimpan.
3. Perintah `bukuPBO.beriDiskon(20000)` memanggil *Method* kedua untuk melakukan perhitungan matematika, yaitu mengurangi atribut harga awal (100.000) dengan nilai potongan (20.000) sehingga hasilnya menjadi 80.000.

1.3 Deklarasi dan Instansiasi Objek

Class hanyalah sebuah cetak biru (seperti formulir kosong). Agar bisa digunakan untuk mengolah data, kita harus menciptakan wujud nyatanya yang disebut Objek. Proses pembuatannya terdiri dari dua tahap yang biasanya digabung dalam satu baris kode:

1. Deklarasi (Menyiapkan Tempat)

Proses membuat variabel baru sesuai tipe *Class*-nya. Ibarat menyiapkan kursi kosong beserta nametag-nya, tapi wujud siswanya belum ada.

2. Instansiasi (Menciptakan Wujud di Memori)

Proses mengalokasikan ruang memori (RAM) agar objek tersebut memiliki wujud fisik. Di Java, proses ini wajib menggunakan kata kunci *new*. Ibarat siswa nyata yang datang dan duduk di kursi tersebut.

Hanya dengan satu *Class*, kita bisa melakukan instansiasi (*new*) berkali-kali untuk menciptakan banyak objek sekaligus.

Tabel 1. 2 Contoh Penerapan Deklarasi dan Instansiasi

```
// 1. Ini adalah Class (Cetakan / Formulir Kosong)
Class Buku {
    String judul;
    String pengarang;
    int harga;

    Buku(String inputJudul, String inputPengarang, int
inputHarga) {
        judul = inputJudul;
        pengarang = inputPengarang;
        harga = inputHarga;
    }

    void tampilkanInfo() {
        System.out.println("Buku " + judul + " karya " +
pengarang + " harganya Rp" + harga);
    }
}

// 2. Ini adalah Program Utama (Tempat Eksekusi)
public Class Main {
    public static void main(String[] args) {

        System.out.println("---      PROSES      DEKLARASI      &
INSTANSIASI ---");

        // Objek 1 (Mencetak buku pertama ke memori)
        Buku bukuSatu = new Buku("Pemrograman Java",
"Budi", 85000);
```

```

        // Objek 2 (Mencetak buku kedua ke memori yang
        berbeda)
        Buku bukuDua = new Buku("Logika Algoritma", "Siti",
        60000);

        System.out.println("--- MEMANGGIL METHOD OBJEK ---
        ");

        // Membuktikan bahwa memori mereka tidak tertukar
        bukuSatu.tampilkanInfo();
        bukuDua.tampilkanInfo();
    }
}

```

Alur Eksekusinya:

1. Saat baris `Buku bukuSatu = new Buku(...)` dieksekusi, komputer memotong ruang di RAM khusus untuk `bukuSatu` dan mengisi datanya dengan "Pemrograman Java".
2. Saat baris `Buku bukuDua = new Buku(...)` dieksekusi, komputer memotong ruang RAM baru yang terpisah untuk `bukuDua` dan mengisi datanya dengan "Logika Algoritma".
3. Karena wujud fisik mereka di memori terpisah, saat `tampilkanInfo()` dipanggil, masing-masing objek akan mencetak data pribadinya sendiri tanpa saling mengganggu. Inilah kehebatan utama Object-Oriented Programming!

1.4 Mengakses Anggota Class

Setelah objek diciptakan di memori, seluruh data dan kemampuannya dibungkus rapat. Untuk menjangkau dan berinteraksi dengan isi bungkusan tersebut, kita menggunakan Operator Titik (.) yang berfungsi ibarat "*remote control*".

1. Mengakses & Memanipulasi Atribut (Data)
 - Rumus: `namaObjek.namaAtribut`
 - Fungsi: Untuk membaca atau mengubah isi data spesifik milik suatu objek. Contoh: `siswaDua.nilaiPBO = 90;` (Mengubah nilai khusus pada memori Budi, tidak berdampak ke Aditya).
2. Memanggil *Method* (Aksi)
 - Rumus: `namaObjek.namaMethod()`
 - Fungsi: Untuk menyuruh objek mengeksekusi instruksi pekerjaannya. Wajib menggunakan tanda kurung (), yang bisa diisi data (parameter) jika diminta.
 - Contoh: `siswaSatu.ikutUjianPraktik(85);`

Tabel 1. 3 Contoh Penerapan Kode Akses Anggota Kelas

```
// 1. Cetaklan Class
Class SiswaSMK {
    // Atribut
    String namaSiswa;
    int nilaiPBO;

    // Constructor
    SiswaSMK(String inputNama) {
        namaSiswa = inputNama;
        nilaiPBO = 0; // Nilai default awal
    }

    // Method Pertama
    void ikutUjianPraktik(int nilaiUjian) {
        nilaiPBO = nilaiUjian;
        System.out.println(namaSiswa + " mendapatkan nilai
ujian: " + nilaiPBO);
    }

    // Method Kedua
    void cekStatusKelulusan() {
        if (nilaiPBO >= 75) {
            System.out.println("Status " + namaSiswa + " :
LULUS");
        } else {
            System.out.println("Status " + namaSiswa + " :
REMIDI");
        }
    }
}

// 2. Program Utama
public Class Main {
    public static void main(String[] args) {

        // Tahap Instansiasi (Menciptakan wujud 2 objek)
        SiswaSMK siswaSatu = new SiswaSMK("Aditya");
        SiswaSMK siswaDua = new SiswaSMK("Budi");

        System.out.println("--- MENGAkses ATRIBUT LANGSUNG
---");
        // Menggunakan titik (.) untuk memanipulasi data
        Budi secara manual
        siswaDua.nilaiPBO = 90;
        System.out.println("Nilai PBO Budi diubah manual
menjadi: " + siswaDua.nilaiPBO);

        System.out.println("\n--- MEMANGGIL METHOD ---");
    }
}
```

```

        // Menggunakan titik (.) untuk menyuruh Aditya
        menjalankan aksi ikut ujian
        siswaSatu.ikutUjianPraktik(85);

        System.out.println("\n--- HASIL AKHIR ---");
        // Menggunakan titik (.) untuk menyuruh kedua objek
        mengecek kelulusan
        siswaSatu.cekStatusKelulusan();
        siswaDua.cekStatusKelulusan();
    }
}

```

Penjelasan Jalur Eksekusi Operator Titik:

1. siswaDua.nilaiPBO = 90;
Sistem akan mencari memori milik siswaDua (yaitu Budi), membuka bungkusan atributnya, dan mengganti nilai PBO-nya dari 0 menjadi 90 secara sepihak. Data Aditya tidak akan terpengaruh.
2. siswaSatu.ikutUjianPraktik(85);
Sistem akan menyuruh objek siswaSatu (Aditya) untuk melakukan aksi ikutUjianPraktik. Angka 85 akan dikirimkan ke dalam parameter untuk diproses oleh *Method* tersebut.
3. siswaDua.cekStatusKelulusan();
Sistem menyuruh Budi untuk mengecek nilainya sendiri. Karena pada baris sebelumnya nilai Budi sudah kita ubah menjadi 90 secara manual, maka output yang keluar untuk Budi adalah LULUS.

1.5 Interaksi Antar Objek

Dalam pemrograman berorientasi objek, sebuah objek sering kali tidak bekerja sendirian. Objek (misalnya Siswa) tidak bisa mengubah datanya sendiri secara sepihak, melainkan membutuhkan objek lain (misalnya Guru) untuk melakukannya. Kunci dari interaksi ini adalah menjadikan sebuah Objek utuh sebagai parameter di dalam *Method* milik Objek lain. Hal ini memungkinkan Objek A untuk mengakses dan mengubah data di dalam Objek B.

Berikut adalah contoh penerapan kode utuh yang mendemonstrasikan bagaimana dua objek berbeda (Siswa dan Guru) saling berinteraksi, di mana objek Guru mengubah data di dalam objek Siswa.

Tabel 1. 4 Contoh Penerapan Interaksi Antar Objek

```

// 1. Cetakan Pertama: Objek yang akan menerima aksi
Class Siswa {

```

```

String nama;
int nilai;

// Constructor Siswa
Siswa(String inputNama) {
    nama = inputNama;
    nilai = 0; // Nilai default sebelum ujian
}

// Method untuk melihat data diri sendiri
void cekRapot() {
    System.out.println("Rapot " + nama + " | Nilai PBO:
" + nilai);
}
}

// 2. Cetakan Kedua: Objek yang akan memberikan aksi
Class Guru {
    String namaGuru;

    // Constructor Guru
    Guru(String inputNama) {
        namaGuru = inputNama;
    }

    // METHOD INTERAKSI: Parameter meminta wujud asli dari
'Siswa'
    void beriNilai(Siswa targetSiswa, int nilaiUjian) {
        // Objek Guru merogoh ke dalam memori Siswa dan
mengubah nilainya
        targetSiswa.nilai = nilaiUjian;
        System.out.println("-> Guru " + namaGuru + "
memasukkan nilai " + nilaiUjian + " untuk " +
targetSiswa.nama);
    }
}

// 3. Program Utama (Tempat Eksekusi)
public Class Main {
    public static void main(String[] args) {

        System.out.println("--- TAHAP 1: MENCETAK OBJEK --
-");

        // Menciptakan dua siswa dan satu guru (mereka belum
saling kenal)
        Siswa muridSatu = new Siswa("Aditya");
        Siswa muridDua = new Siswa("Budi");
        Guru guruPBO = new Guru("Pak Saiful");

        // Cek rapot sebelum interaksi
        muridSatu.cekRapot();
    }
}

```

```

        muridDua.cekRapot();

        System.out.println("\n--- TAHAP 2: PROSES INTERAKSI
OBJEK ---");
        // Objek Guru memanggil Method-nya, dan menangkap
objek Siswa ke dalam parameternya
        guruPBO.beriNilai(muridSatu, 95);
        guruPBO.beriNilai(muridDua, 75);

        System.out.println("\n--- TAHAP 3: PEMBUKTIAN HASIL
INTERAKSI ---");
        // Membuktikan bahwa nilai di dalam memori siswa
telah berhasil diubah oleh guru
        muridSatu.cekRapot();
        muridDua.cekRapot();
    }
}

```

Penjelasan Singkat Alur Eksekusi:

1. Tahap 1: Tiga objek tercipta di memori secara mandiri. Nilai Aditya dan Budi saat ini masih murni 0.
2. Tahap 2: Di sinilah keajaiban interaksi terjadi. Perintah `guruPBO.beriNilai(muridSatu, 95);` membuat *Method* di dalam kelas Guru aktif. *Method* tersebut menerima wujud fisik `muridSatu`, lalu mengakses atribut nilai di dalamnya menggunakan operator titik (`targetSiswa.nilai`), dan menggantinya menjadi 95.
3. Tahap 3: Saat Aditya memanggil kemampuannya sendiri (`muridSatu.cekRapot()`), angka yang keluar sudah bukan 0 lagi, melainkan 95. Ini membuktikan data Aditya berhasil dimanipulasi oleh objek Guru dari luar!

BAB 2

ENKAPSULASI

2.1 Konsep Data Hiding

Mengakses dan mengubah data secara langsung menggunakan operator titik (`murid1.nilai = 95`) memang praktis. Namun, di industri profesional, membiarkan data terbuka bebas adalah celah keamanan fatal. Standar pemrograman mewajibkan data diisolasi dengan ketat, sebuah konsep yang disebut *Data Hiding* (Penyembunyian Data).

Tujuan dari *Data Hiding* adalah menjaga Integritas Data (keabsahan nilai). Jika variabel dibiarkan terbuka (*public*), *Class* eksternal bisa mengubah datanya sesuka hati. Hal ini memungkinkan masuknya nilai tidak masuk akal (seperti nyawa *player* -50 atau saldo minus), yang akan memicu *bug* atau eksploitasi oleh peretas.

Bayangkan sebuah minimarket:

- Tanpa *Data Hiding* (Laci Terbuka): Pembeli mengambil kembaliannya sendiri dari mesin kasir. Risiko pencurian dan kesalahan hitung sangat tinggi.
- Penerapan *Data Hiding* (Laci Terkunci): Laci uang (Atribut) dikunci rapat. Jika pembeli butuh kembalian, ia harus memintanya lewat sang Kasir (*Method*). Kasir akan memvalidasi transaksinya terlebih dahulu sebelum mengeluarkan uang.

Bahaya Tanpa *Data Hiding*

Jika kita membuat *Class* AkunGame dengan atribut saldoDiamond yang terbuka, akan terjadi dua masalah fatal:

- Modifikasi Ilegal: *Class* luar bisa melakukan bypass nilai (contoh: `akunPlayer.saldoDiamond = 50000`;) tanpa melalui proses top-up yang sah.
- Invalid State (Data Tidak Wajar): Sistem tidak punya pertahanan jika *Class* luar memasukkan nilai tidak masuk akal (contoh: `akunPlayer.saldoDiamond = -999`;) . Mesin akan pasrah menerima data tersebut dan merusak logika permainan.

Berikut adalah contoh penerapan kode yang benar untuk mengatasi masalah keamanan pada AkunGame dengan menggunakan konsep *Data Hiding* (Enkapsulasi).

Untuk mengunci data, kita menggunakan kata kunci `private` pada atribut, lalu menyediakan *Method* khusus (sebagai "Kasir") untuk mengubah atau melihat data tersebut secara aman.

Tabel 2. 1 Contoh Penerapan *Data Hiding*

```
// 1. Class dengan Data Hiding (Enkapsulasi)
Class AkunGame {

    // ATRIBUT DIKUNCI (Hanya bisa diakses dari dalam Class
    ini sendiri)
    private String username;
    private int saldoDiamond;

    // Constructor
    AkunGame(String inputUsername, int topupAwal) {
        username = inputUsername;
        // Validasi awal saat objek dibuat
        if (topupAwal >= 0) {
            saldoDiamond = topupAwal;
        } else {
            saldoDiamond = 0; // Mencegah nilai minus di
awal
        }
    }

    // METHOD KASIR 1: Untuk menambah saldo (Setter /
    Modifikator)
    public void topUpDiamond(int jumlahTopUp) {
        // Logika Validasi (Sistem Pertahanan)
        if (jumlahTopUp > 0) {
            saldoDiamond = saldoDiamond + jumlahTopUp;
            System.out.println("Top-up " + jumlahTopUp + "
berhasil! Saldo bertambah.");
        } else {
            // Menolak input tidak masuk akal (seperti
minus)
            System.out.println("GAGAL: Jumlah top-up tidak
valid! Sistem menolak transaksi.");
        }
    }

    // METHOD KASIR 2: Untuk melihat saldo (Getter)
    public int cekSaldo() {
        return saldoDiamond;
    }
}

// 2. Program Utama (Tempat Eksekusi)
public Class Main {
    public static void main(String[] args) {

        System.out.println("--- MEMBUAT AKUN BARU ---");
        AkunGame akunPlayer = new AkunGame("ProPlayer99",
100);
    }
}
```

```

        // akunPlayer.saldoDiamond = 50000; // <- JIKA
DIBUKA, BARIS INI AKAN ERROR MERAH! (Akses Ditolak)

        System.out.println("\n--- PERCOBAAN TOP-UP SAH ---
");
        // Cara yang benar: Meminta Method untuk memproses
data
        akunPlayer.topUpDiamond(50000);

        System.out.println("\n--- PERCOBAAN HACKING (NILAI
MINUS) ---");
        // Mencoba merusak sistem dengan nilai negatif
        akunPlayer.topUpDiamond(-999);

        System.out.println("\n--- CEK HASIL AKHIR ---");
        // Cara yang benar untuk melihat nilai
        System.out.println("Sisa Diamond saat ini: " +
akunPlayer.cekSaldo());
    }
}

```

Penjelasan Eksekusi Kode

1. Mengunci Atribut (private)

Dengan menambahkan kata *private* di depan `saldoDiamond`, atribut tersebut kini terkunci rapat di dalam brankas `AkunGame`. Jika *Class* `Main` (atau hacker) mencoba memaksa masuk menggunakan `akunPlayer.saldoDiamond = 50000`, program akan langsung *Error* dan menolak untuk berjalan.

2. Membangun Sistem Pertahanan di dalam *Method*

Karena data sudah dikunci, interaksi hanya bisa dilakukan melalui *Method* `topUpDiamond`. Di sinilah kita memasang "satpam" berupa logika *if* (`jumlahTopUp > 0`).

Saat objek menerima angka 50000, satpam mengizinkan data masuk dan saldo bertambah.

Saat objek diserang dengan angka -999, satpam memblokirnya dan mencetak pesan "GAGAL", sehingga saldo asli pemain tetap aman.

3. *Method* Getter (`cekSaldo`)

Karena laci dikunci, kita juga tidak bisa sembarangan melihat isinya. Kita harus menggunakan *Method* `cekSaldo()` yang bertugas secara spesifik hanya untuk melaporkan angka saldo saat ini ke luar sistem, tanpa memberikan izin untuk mengubahnya.

2.2 Access Modifier

Dalam Pemrograman Berorientasi Objek (PBO), *Access Modifier* berfungsi sebagai "gembok" keamanan untuk mengatur siapa saja yang diizinkan melihat atau mengubah data di dalam sebuah *Class*. Hal ini sangat penting untuk mencegah kerusakan data dari modifikasi eksternal (*Data Hiding*).

Terdapat 3 level hak akses utama, dari yang paling terbuka hingga paling tertutup:

1. *Public* (Terbuka Bebas)
 - Sifat: Data atau *Method* bisa diakses, dibaca, dan diubah dari *Class* mana saja secara bebas.
 - Analogi: Papan pengumuman sekolah yang bisa dibaca oleh semua orang.
 - Hasil: Eksekusi dari *Class* luar selalu berhasil tanpa hambatan.
2. *Protected* (Keluarga / Turunan)
 - Sifat: Data hanya bisa diakses oleh *Class* itu sendiri, *Class* turunannya (*Inheritance*), atau *Class* lain dalam satu folder (*package*) yang sama.
 - Analogi: Fasilitas lab komputer yang hanya boleh diakses oleh siswa dari jurusan terkait.
3. *Private* (Tertutup Rapat / Rahasia)
 - Sifat: Level keamanan tertinggi. Data terisolasi secara mutlak dan hanya bisa diakses dari dalam *Class* itu sendiri.
 - Analogi: Isi dompet atau chat pribadi yang tidak boleh dibuka oleh orang lain.
 - Hasil: Jika *Class* luar (seperti file *Main*) mencoba mengakses atau memanipulasi data ini, program akan langsung diblokir dan menampilkan pesan *Error*.

Tabel 2. 2 Contoh Penerapan Access Modifier

```
// 1. Class Cetakan dengan berbagai tingkat keamanan
Class AkunSiswa {

    // PUBLIC: Terbuka bebas untuk dibaca dan diubah dari
    Class mana saja
    public String namaSiswa;

    // PROTECTED: Terbuka untuk turunan atau Class dalam
    satu folder (package) yang sama
    protected String jurusan;

    // PRIVATE: Tertutup rapat (Data Hiding), murni hanya
    bisa diakses dari dalam Class ini
    private int saldoKantin;
```

```

        // Constructor untuk mengisi data awal
        AkunSiswa(String inputNama, String inputJurusan, int
inputSaldo) {
            namaSiswa = inputNama;
            jurusan = inputJurusan;
            saldoKantin = inputSaldo;
        }

        // Method Public: Sebagai "Kasir" untuk mengakses data
Private secara aman
        public void cekSaldo() {
            System.out.println("Saldo Kantin " + namaSiswa +
" : Rp" + saldoKantin);
        }
    }

// 2. Program Utama (Class Luar sebagai tempat eksekusi)
public Class Main {
    public static void main(String[] args) {

        // Membuat objek baru di memori
        AkunSiswa siswal = new AkunSiswa("Aditya", "RPL",
50000);

        System.out.println("--- UJI COBA HAK AKSES DARI
LUAR CLASS ---");

        // 1. EKSEKUSI PUBLIC (BERHASIL)
        // Class Main diizinkan memanipulasi atribut public
secara langsung
        siswal.namaSiswa = "Aditya Pratama";
        System.out.println("Nama berhasil diubah menjadi:
" + siswal.namaSiswa);

        // 2. EKSEKUSI PROTECTED (BERHASIL)
        // Class Main diizinkan mengubahnya karena
diasumsikan berada dalam folder yang sama
        siswal.jurusan = "TKJ";
        System.out.println("Jurusan berhasil diubah
menjadi: " + siswal.jurusan);

        // 3. EKSEKUSI PRIVATE (AKAN DIBLOKIR / ERROR)
        // Jika dua baris di bawah ini tanda komentarnya
(//) dihapus, program akan ERROR MERAH!
        // Mesin Java menolak keras Class Main mengubah
atau melihat variabel private secara langsung.

        // siswal.saldoKantin = 1000000;
        // System.out.println(siswal.saldoKantin);
    }
}

```

```

        System.out.println("\n--- CARA AMAN MENGAKSES DATA
PRIVATE ---");
        // Cara yang SAH dan BENAR adalah menyuruh Method
public dari dalam objek tersebut
        // untuk melaporkan datanya ke luar.
        siswa1.cekSaldo();
    }
}

```

Ringkasan Alur Eksekusi Keamanan:

1. Variabel namaSiswa (Public) dan jurusan (Protected) sangat mudah ditembus oleh operator titik (.) dari *Class Main*. Keduanya berhasil diubah nilainya tanpa penolakan dari sistem.
2. Variabel saldoKantin (Private) terkunci dengan sempurna. *Class Main* sama sekali tidak bisa menyentuhnya menggunakan siswa1.saldoKantin.
3. Karena datanya digembok secara private, satu-satunya celah agar *Class* luar bisa melihat jumlah saldo tersebut adalah dengan memanggil *Method* siswa1.cekSaldo();, di mana *Method* tersebut sudah diatur sebagai public oleh programmer.

2.3 Method Getter dan Setter

Dalam penerapan *Data Hiding*, atribut kelas dikunci dengan hak akses *private*. Agar data tersebut tetap bisa dibaca atau diubah dari luar *Class* secara sah, kita wajib menyediakan "pintu resmi" berupa *Method* public yang dikenal sebagai *Getter* dan *Setter*.

1. Method Getter (Pembaca)

- Fungsi: Membaca dan mengembalikan nilai dari atribut yang di-private.
- Aturan: Biasanya namanya diawali kata get (misal: getSaldoKantin()). *Method* ini tidak boleh bertipe void karena harus melempar data keluar, sehingga wajib menggunakan perintah return.

2. Method Setter (Pengubah)

- Fungsi: Menerima data/input dari luar untuk dimasukkan ke dalam atribut yang di-private.
- Aturan: Biasanya namanya diawali kata set (misal: setTopUpSaldo()). *Method* ini bertipe void (tidak mengembalikan nilai), tetapi wajib memiliki parameter sebagai jalur masuk data.

Analogi Bank (Brankas dan Teller)

Anggap variabel private adalah brankas bank yang terkunci.

Getter adalah saat kalian bertanya ke Teller berapa sisa saldo kalian, lalu Teller mengecek brankas dan melapor kembali ke kalian. *Setter* adalah saat kalian menyetorkan uang ke Teller, lalu Teller yang membukakan brankas dan memasukkan uang tersebut secara aman.

Tabel 2. 3 Contoh Penerapan *Method Getter* dan *Setter*

```
// 1. Class Cetakan dengan Data Tersembunyi
Class AkunSiswa {

    // Atribut public (bisa diakses langsung)
    public String namaSiswa;

    // Atribut private (brankas terkunci, tidak bisa
    diakses langsung)
    private int saldoKantin;

    // Constructor untuk mengisi data awal
    AkunSiswa(String inputNama, int isiSaldo) {
        namaSiswa = inputNama;
        saldoKantin = isiSaldo;
    }

    // =====
    // METHOD GETTER (Pintu Resmi untuk Membaca)
    // =====
    public int getSaldoKantin() {
        // Melaporkan isi brankas ke luar
        return saldoKantin;
    }

    // =====
    // METHOD SETTER (Pintu Resmi untuk Mengubah)
    // =====
    public void setTopUpSaldo(int tambahanSaldo) {
        // Menerima uang dari luar via parameter, lalu
        memprosesnya ke dalam brankas
        if (tambahanSaldo > 0) {
            saldoKantin = saldoKantin + tambahanSaldo;
            System.out.println("Transaksi berhasil!
Saldo bertambah Rp" + tambahanSaldo);
        } else {
            System.out.println("Transaksi      Gagal:
Nominal top-up tidak valid.");
        }
    }
}

// 2. Program Utama (Tempat Eksekusi)
public Class Main {
    public static void main(String[] args) {
```

```

        System.out.println("--- MEMBUAT AKUN BARU ---
");
        AkunSiswa siswa1 = new AkunSiswa("Aditya",
50000);

        // siswa1.saldoKantin = 20000; // <- JIKA
DIBUKA, INI AKAN ERROR KARENA PRIVATE!

        System.out.println("\n---      PROSES      SETTER
(MENGUBAH DATA) ---");
        // Cara yang SAH untuk menambah saldo adalah
melempar angkanya ke dalam Setter
        siswa1.setTopUpSaldo(20000);

        // Mencoba top-up dengan angka tidak wajar
(akan ditahan oleh logika di dalam Setter)
        siswa1.setTopUpSaldo(-5000);

        System.out.println("\n---      PROSES      GETTER
(MEMBACA DATA) ---");
        // Cara yang SAH untuk melihat sisa saldo
adalah memanggil Method Getter
        System.out.println("Nama Siswa      : " +
siswa1.namaSiswa); // Public, bisa langsung cetak
        System.out.println("Saldo Terkini : Rp" +
siswa1.getSaldoKantin()); // Private, wajib pakai
Getter
    }
}

```

Penjelasan Singkat Alur Eksekusi:

1. Perlindungan Data: Karena `saldoKantin` bersifat *private*, kita terhindar dari keteledoran mengubah data secara langsung.
2. Proses *Setter*: Saat perintah `siswa1.setTopUpSaldo(20000);` dipanggil, uang dilempar ke "Teller". Teller akan mengecek apakah uangnya valid (lebih dari 0). Karena valid, uang ditambahkan ke brankas. Saat mencoba memasukkan -5000, Teller menolak dan saldo tetap aman.
3. Proses *Getter*: Saat ingin mencetak saldo di baris terakhir, kita memanggil `siswa1.getSaldoKantin()`. *Method* ini akan merogoh ke dalam brankas, mengambil angka terbaru (70000), dan mengembalikannya (return) untuk dicetak di layar.

BAB 3

INHERITANCE (PEWARISAN)

3.1 Konsep Pewarisan Sifat

Saat membangun aplikasi besar, menulis ulang atribut dan *Method* yang sama (seperti nama dan alamat) di puluhan *Class* yang berbeda akan membuat kode menjadi redundan, berantakan, dan tidak efisien. Untuk mengatasinya, standar industri mewajibkan penggunaan konsep Pewarisan (*Inheritance*).

Pewarisan adalah teknik di mana sebuah *Class* baru diciptakan dengan cara "mewarisi" seluruh atribut dan *Method* dari *Class* yang sudah ada.

1. *SuperClass* (Kelas Induk): *Class* tingkat atas yang menyimpan data dan perilaku yang bersifat umum (contoh: Pegawai).
2. *SubClass* (Kelas Anak): *Class* turunan yang posisinya di bawah. Ia otomatis mendapatkan "warisan" gratis (semua atribut & *Method*) dari Induknya, ditambah ia bisa memiliki kekhususannya sendiri (contoh: Guru punya *Method* mengajar(), StafTU punya rekapAbsen()).

Dalam Java, tali pewarisan dihubungkan menggunakan kata kunci *extends*.

Rumus: *Class* NamaAnak *extends* NamaInduk { ... }

Saat mesin membaca *extends*, ia langsung menggandakan seluruh isi Induk ke dalam Anak secara otomatis tanpa perlu diketik ulang.

Atribut yang di-*private* oleh Induk tidak akan pernah diwariskan ke Anaknya. Untuk mengatasinya, kita menggunakan level akses *protected*. Jika menggunakan *protected*, atribut akan aman dari jangkauan *Class* asing, namun anak turunannya (*SubClass*) mendapat jalur khusus (VVIP) untuk mewarisi dan menggunakannya.

Alih-alih menulis atribut nama dan nip berkali-kali di *Class* Guru, StafTU, dan KepalaSekolah, kita cukup menuliskannya satu kali saja di *SuperClass* Pegawai. Jika ada perubahan sistem (misal: tambah golongan darah), kita cukup menambahkannya di Pegawai, dan seluruh anak buahnya akan otomatis memilikinya!

Berikut adalah contoh penerapan kode utuh yang mendemonstrasikan konsep Pewarisan (*Inheritance*) menggunakan kata kunci *extends* dan hak akses *protected*.

Tabel 3. 1 Contoh Penerapan Inheritance

```
// 1. SUPERCLASS (Kelas Induk)
Class Pegawai {

    // Menggunakan protected agar bisa diwariskan khusus ke
    anak-anaknya (SubClass),
    // namun tetap terkunci dari Class asing di luar sistem.
    protected String nama;
```

```

        protected String nip;

        // Method umum yang bisa dipakai oleh semua jenis pegawai
        public void absenHadir() {
            System.out.println("Pegawai " + nama + " telah melakukan absensi kehadiran.");
        }
    }

    // 2. SUBCLASS (Kelas Anak)
    // Kata kunci 'extends' mengaktifkan tali pewarisan dari Induk (Pegawai) ke Anak (Guru)
    Class Guru extends Pegawai {

        // Atribut spesifik yang hanya dimiliki oleh Guru
        public String mataPelajaran;

        // Method spesifik yang hanya dimiliki oleh Guru
        public void mengajar() {
            // Objek Guru bisa langsung menyebut variabel 'nama' seolah miliknya sendiri
            System.out.println("Guru " + nama + " masuk kelas untuk mengajar " + mataPelajaran + ".");
        }
    }

    // 3. PROGRAM UTAMA (Tempat Eksekusi)
    public Class Main {
        public static void main(String[] args) {

            System.out.println("--- TAHAP PENGISIAN DATA ---");
            // Mencetak objek nyata dari cetakan Guru
            Guru guruPBO = new Guru();

            // BUKTI PEWARISAN 1: Objek guru bisa mengisi atribut nama dan nip
            // hasil warisan gratis dari Class Pegawai.
            guruPBO.nama = "Pak Saiful";
            guruPBO.nip = "198001012005011001";

            // Mengisi atribut miliknya sendiri
            guruPBO.mataPelajaran = "PBO";

            System.out.println("Data berhasil diisi!");
            System.out.println("\n--- TAHAP EKSEKUSI METHOD ---");

            // BUKTI PEWARISAN 2: Objek guru bisa menjalankan aksi absenHadir()

```

```

        // yang secara fisik kodenya hanya ada di Class
Pegawai.
        guruPBO.absenHadir();

        // Menjalankan aksinya sendiri
        guruPBO.mengajar();
    }
}

```

Penjelasan Singkat Alur Bukti Pewarisan:

1. Kode Lebih Bersih: Perhatikan di dalam *Class* Guru, kita sama sekali tidak mengetik ulang variabel nama dan nip.
2. Pemberian Hak Akses: Karena atribut di induk disetel sebagai *protected*, *Class* anak mendapat akses khusus. Ini terlihat pada *Method* mengajar(), di mana *Class* Guru bisa memanggil variabel nama tanpa memicu *error*.
3. Pemanfaatan Warisan: Saat di Program Utama, objek guruPBO bisa langsung menggunakan guruPBO.nama dan guruPBO.absenHadir(). Mesin Java paham bahwa Guru adalah "anak" dari Pegawai, sehingga ia berhak atas semua fasilitas milik induknya.

3.2 Pemanggilan Konstruktor Induk

Dalam implementasi PBO di dunia nyata, pengisian data objek dilakukan secara instan melalui *Constructor*. Ketika *SuperClass* (Induk) dan *SubClass* (Anak) masing-masing memiliki *Constructor*, *SubClass* bertugas menerima seluruh data awal, namun wajib "menyetorkan" data dasar ke Induknya. Jembatan penyetoran data ini adalah perintah *super()*.

Analogi Pendaftaran Ulang

- Panitia Pusat (Induk): Bertugas memproses data dasar (Nama, NISN).
- Panitia Jurusan (Anak): Bertugas memproses data spesifik jurusan.
- Saat siswa mendaftar ke meja jurusan, panitia jurusan akan meminta semua data, namun wajib lapor dan mengirim data dasar ke pusat terlebih dahulu (menggunakan *super()*) sebelum memproses data jurusannya sendiri.

Aturan Mutlak *super()*

- Wajib di Baris Pertama: Pemanggilan *super()* harus diletakkan di baris paling atas pada *Constructor SubClass*. Induk harus selalu diutamakan dan dieksekusi lebih dulu.
- Menghubungkan Parameter: *SubClass* harus melempar variabel yang dibutuhkan oleh Induk ke dalam tanda kurung *super(...)*.

Berikut adalah contoh penerapan kode utuh yang mendemonstrasikan bagaimana *SubClass* menggunakan kata kunci *super()* untuk mengirim data ke *Constructor SuperClass*.

Tabel 3. 2 Contoh Penerapan Pemanggilan Konstruktor Induk

```
// 1. SUPERCLASS (Induk / Panitia Pusat)
Class Pegawai {
    protected String nama;
    protected String nip;

    // Constructor Induk: Bertugas memproses data dasar
    Pegawai(String inputNama, String inputNip) {
        nama = inputNama;
        nip = inputNip;
        System.out.println("-> [Pusat] Data dasar " + nama
+ " berhasil disimpan.");
    }
}

// 2. SUBCLASS (Anak / Panitia Jurusan)
Class Guru extends Pegawai {
    public String mataPelajaran;

    // Constructor Anak: Menerima SEMUA data dari Main
    Guru(String inputNama, String inputNip, String
inputMapel) {

        // WAJIB DI BARIS PERTAMA: Melempar data dasar ke
Induk terlebih dahulu
        super(inputNama, inputNip);

        // Setelah Induk selesai, barulah Anak memproses
datanya sendiri
        mataPelajaran = inputMapel;
        System.out.println("-> [Jurusan] Data mapel " +
mataPelajaran + " berhasil disimpan.\n");
    }

    public void tampilkanProfil() {
        System.out.println("Nama Guru : " + nama);
        System.out.println("NIP          : " + nip);
        System.out.println("Mengajar   : " + mataPelajaran);
        System.out.println("-----");
    }
}

// 3. PROGRAM UTAMA (Tempat Eksekusi)
public Class Main {
    public static void main(String[] args) {
```

```

        System.out.println("--- PROSES PENDAFTARAN GURU 1
        ---");
        // Main melempar 3 data sekaligus ke Constructor
        Guru
        Guru guruSatu = new Guru("Pak Saiful", "19800101",
        "PBO");

        System.out.println("--- PROSES PENDAFTARAN GURU 2
        ---");
        // Main melempar 3 data sekaligus ke Constructor
        Guru
        Guru guruDua = new Guru("Bu Ratna", "19850202",
        "Basis Data");

        System.out.println("--- HASIL AKHIR ---");
        guruSatu.tampilkanProfil();
        guruDua.tampilkanProfil();
    }
}

```

Penjelasan Singkat Alur Eksekusi:

- Saat baris `new Guru("Pak Saiful", "19800101", "PBO")` dijalankan, ketiga data tersebut masuk ke dalam *Constructor* *Guru*.
- Di dalam *Constructor* *Guru*, perintah `super(inputNama, inputNip);` langsung menangkap "Pak Saiful" dan "19800101", lalu melemparkannya naik ke *Constructor* *Pegawai*.
- *Constructor* *Pegawai* (Induk) bekerja lebih dulu menyimpan nama dan NIP, lalu mencetak pesan [Pusat].
- Setelah Induk selesai, kendali kembali turun ke *Constructor* *Guru* (Anak) untuk menyimpan "PBO" ke dalam *mataPelajaran* dan mencetak pesan [Jurusan].

3.3 Hierarki Pewarisan Bertingkat

Dalam sistem yang lebih kompleks, sebuah *SubClass* (Anak) bisa bertindak kembali sebagai *SuperClass* (Induk) bagi *Class* lain di bawahnya. Hal ini menciptakan sebuah rantai pewarisan yang panjang.

Analogi Pohon Silsilah (Kakek - Bapak - Anak)

1. *Pegawai* (Kakek): Memegang data paling dasar (nama, nip).
2. *Guru* (Bapak): Mewarisi *Pegawai*, lalu menambah data spesifiknya (*mataPelajaran*).
3. *GuruWaliKelas* (Anak): Mewarisi *Guru*, lalu menambah data spesifik tugasnya (*kelasBina*).

4. Keuntungan: *Class* *GuruWaliKelas* secara otomatis memborong seluruh atribut dan *Method* dari "Bapak" dan "Kakeknya" tanpa perlu menulis ulang satu baris kode pun.

Rantai Kata Kunci *extends*

Pewarisan diaktifkan secara berurutan:

- *Class* *Guru* *extends* *Pegawai*
- *Class* *GuruWaliKelas* *extends* *Guru*

Berikut adalah contoh penerapan kode utuh yang mendemonstrasikan Pewarisan Bertingkat (*Multilevel Inheritance*) beserta estafet data menggunakan *Constructor* dan *super()*.

Tabel 3. 3 Contoh Penerapan Pewarisan Bertingkat

```
// 1. KAKEK (SuperClass Paling Atas)
Class Pegawai {
    protected String nama;
    protected String nip;

    // Constructor Kakek: Menerima 2 data dasar
    Pegawai(String inputNama, String inputNip) {
        nama = inputNama;
        nip = inputNip;
        System.out.println("-> [Kakek] Data Pegawai (" +
nama + ") tersimpan.");
    }
}

// 2. BAPAK (SubClass dari Pegawai)
Class Guru extends Pegawai {
    protected String mataPelajaran;

    // Constructor Bapak: Menerima 3 data
    Guru(String inputNama, String inputNip, String
inputMapel) {
        // TAHAP 3 & 4: Melempar 2 data dasar ke Kakek
        terlebih dahulu
        super(inputNama, inputNip);

        // Setelah Kakek selesai, Bapak menyimpan datanya
        sendiri
        mataPelajaran = inputMapel;
        System.out.println("-> [Bapak] Data Guru Mapel (" +
mataPelajaran + ") tersimpan.");
    }
}

// 3. ANAK (SubClass dari Guru)
```

```

Class GuruWaliKelas extends Guru {
    private String kelasBina;

    // Constructor Anak: Menerima SEMUA (4) data dari Main
    GuruWaliKelas(String inputNama, String inputNip, String
inputMapel, String inputKelas) {
        // TAHAP 1 & 2: Melempar 3 data ke Bapak terlebih
dahulu
        super(inputNama, inputNip, inputMapel);

        // Setelah Bapak (dan Kakek) selesai, Anak menyimpan
datanya sendiri
        kelasBina = inputKelas;
        System.out.println("-> [Anak] Data Wali Kelas (" +
kelasBina + ") tersimpan.\n");
    }

    // Method pembuktian: Anak bisa memanggil data nama dan
nip milik Kakeknya,
    // serta mataPelajaran milik Bapaknya.
    public void tampilkanInfoWali() {
        System.out.println("Nama Wali Kelas : " + nama);
        System.out.println("NIP           : " + nip);
        System.out.println("Guru  Mapel           : " +
mataPelajaran);
        System.out.println("Membina  Kelas           : " +
kelasBina);
        System.out.println("-----
---");
    }
}

// 4. PROGRAM UTAMA (Tempat Eksekusi)
public Class Main {
    public static void main(String[] args) {

        System.out.println("===  PROSES  INSTANSIASI  WALI
KELAS 1 ===");
        // Mengirim 4 data sekaligus ke Constructor Anak
(GuruWaliKelas)
        GuruWaliKelas waliRPL = new GuruWaliKelas("Pak
Budi", "19850101", "Basis Data", "XII RPL 1");

        System.out.println("===  PROSES  INSTANSIASI  WALI
KELAS 2 ===");
        // Mengirim 4 data sekaligus ke Constructor Anak
(GuruWaliKelas)
        GuruWaliKelas waliTKJ = new GuruWaliKelas("Bu
Siti", "19900202", "Jaringan", "XI TKJ 2");
    }
}

```

```
        System.out.println("===  HASIL  AKHIR  PEMBUKTIAN  
WARISAN ===");  
        // Memanggil Method untuk membuktikan semua data  
        berhasil tergabung  
        waliRPL.tampilkanInfoWali();  
        waliTKJ.tampilkanInfoWali();  
    }  
}
```

Penjelasan Alur Eksekusi (Melihat Estafet Data):

1. [Kakek] akan mencetak pesan pertama kali (karena `super()` memaksa program memprioritaskan Induk tertinggi).
2. [Bapak] mencetak pesan kedua (setelah tugas kakek selesai).
3. [Anak] mencetak pesan terakhir, menandakan seluruh rantai pewarisan dan pembagian tugas telah selesai dengan sempurna.

BAB 4

POLIMORFISME

4.1 Pengantar Polimorfisme

Dalam Pemrograman Berorientasi Objek (PBO), Polimorfisme berasal dari kata *Poly* (banyak) dan *Morph* (bentuk). Ini adalah kemampuan sebuah objek untuk merespons instruksi yang sama dengan cara atau aksi yang berbeda-beda, tergantung pada siapa wujud asli objek tersebut .

Bayangkan Kepala Sekolah memberi satu perintah: "Kerjakan tugas praktik!"

- Siswa RPL merespons dengan: Membuka laptop dan mulai coding.
- Siswa TKJ merespons dengan: Memotong kabel LAN dan setting router. Perintahnya sama, tapi bentuk eksekusinya berbeda sesuai jurusannya.

Mekanisme Utama Polimorfisme

- *Overriding* (@Override): Teknik di mana *Class* Anak tidak pasrah menerima warisan *Method* Induk begitu saja, melainkan menulis ulang *Method* tersebut dengan aksi spesifik miliknya sendiri.
- *Instansiasi Polimorfik*: Teknik membuat objek menggunakan tipe data *SuperClass* (Induk), tetapi wujud fisiknya diisi oleh *SubClass* (Anak). Contoh: Siswa siswaProgrammer = new SiswaRPL();

Berikut adalah implementasi kode utuh dari analogi siswa SMK. Perhatikan bagaimana *Method* yang namanya sama persis bisa menghasilkan output yang berbeda.

Tabel 4. 1 Contoh Penerapan Polimorfisme

```
// 1. SUPERCLASS (Induk yang menyediakan instruksi dasar)
Class Siswa {
    public void kerjakanTugas() {
        System.out.println("Siswa sedang mengerjakan
tugas umum sekolah.");
    }
}

// 2. SUBCLASS RPL (Anak yang menulis ulang perintah Induk)
Class SiswaRPL extends Siswa {
    // @Override adalah tanda bahwa Method ini
    menimpa/mengganti milik Induknya
    @Override
    public void kerjakanTugas() {
        System.out.println("Siswa RPL mengerjakan tugas:
Ngoding aplikasi Java.");
    }
}
```

```
// 3. SUBCLASS TKJ (Anak yang juga menulis ulang perintah
Induk)
Class SiswaTKJ extends Siswa {
    @Override
    public void kerjakanTugas() {
        System.out.println("Siswa TKJ mengerjakan tugas:
Crimping kabel LAN dan setting Mikrotik.");
    }
}

// 4. PROGRAM UTAMA (Tempat Eksekusi)
public Class Main {
    public static void main(String[] args) {

        System.out.println("---      TAHAP      INSTANSIASI
POLIMORFIK ---");
        // Tipe datanya 'Siswa' (Induk), tapi wujud
nyatanya adalah 'Siswa Anak'
        Siswa siswaUmum = new Siswa();
        Siswa siswaProgrammer = new SiswaRPL();
        Siswa siswaJaringan = new SiswaTKJ();

        System.out.println("\n---      INSTRUKSI:      KERJAKAN
TUGAS ---");
        // PEMBUKTIAN: Memanggil Method dengan nama yang
sama persis
        siswaUmum.kerjakanTugas();
        siswaProgrammer.kerjakanTugas(); // Mesin akan
mengeksekusi versi RPL
        siswaJaringan.kerjakanTugas(); // Mesin akan
mengeksekusi versi TKJ
    }
}
```

Penjelasan Alur Kode:

Saat kalian melihat baris kode di Program Utama, kita memanggil `kerjakanTugas()` sebanyak tiga kali dengan ejaan yang persis sama.

Kehebatannya, mesin Java cukup pintar untuk melihat siapa wujud asli objek tersebut di dalam memori. Karena `siswaProgrammer` wujud aslinya adalah `SiswaRPL`, maka yang dieksekusi adalah aksi coding. Karena `siswaJaringan` wujud aslinya adalah `SiswaTKJ`, maka yang dieksekusi adalah aksi memotong kabel. Inilah inti dari Polimorfisme!

4.2 Upcasting dan Downcasting

Upcasting (Pengelompokan Massal), Memasukkan objek *SubClass* (Anak) ke dalam variabel bertipe *SuperClass* (Induk) secara otomatis. Tujuannya agar entitas yang berbeda (Guru dan Siswa) bisa ditampung dalam satu array yang

sama. Kekurangannya, sistem akan mengalami "amnesia" terhadap *Method* spesifik milik Anak.

Downcasting (Mengembalikan Wujud) adalah Proses membuka kembali label Induk dan memaksa objek kembali ke tipe Anak aslinya. Hal ini dilakukan agar kita bisa kembali menggunakan *Method* spesifik (seperti mengajar() atau ujian()). Proses *Downcasting* sangat berisiko memicu *error* (*ClassCastException*) jika wujud aslinya salah ditebak. Oleh karena itu, wajib menggunakan operator *instanceof* sebagai sensor untuk memastikan wujud asli objek sebelum di-*downcast*.

Berikut adalah kode yang mendemonstrasikan bagaimana ratusan user bisa dikelola dalam satu daftar hadir, namun tetap bisa mengeksekusi tugas spesifik mereka dengan aman.

Tabel 4. 2 Contoh Penerapan Upcasting dan Downcasting

```
// 1. SUPERCLASS (Induk)
Class AnggotaSekolah {
    public void absen() {
        System.out.println("Anggota sekolah hadir di platform Resolva.");
    }
}

// 2. SUBCLASS (Anak 1)
Class Guru extends AnggotaSekolah {
    @Override
    public void absen() {
        System.out.println("Guru telah login dan hadir.");
    }

    // Method Spesifik
    public void mengajar() {
        System.out.println("-> Aksi Khusus: Guru mulai membuka kelas Pemrograman Java.");
    }
}

// 3. SUBCLASS (Anak 2)
Class Siswa extends AnggotaSekolah {
    @Override
    public void absen() {
        System.out.println("Siswa SMKN 1 Surabaya telah hadir.");
    }

    // Method Spesifik
    public void ujian() {
```

```

        System.out.println("-> Aksi Khusus: Siswa
mengerjakan tugas praktikum PBL.");
    }
}

// 4. PROGRAM UTAMA
public Class Main {
    public static void main(String[] args) {

        System.out.println("--- TAHAP 1: UPCASTING MASSAL
---");
        // Membuat wadah Induk yang bisa menampung
semuanya
        AnggotaSekolah[] daftarHadir = new
AnggotaSekolah[2];

        // Upcasting: Wujud asli Anak dibungkus ke dalam
array Induk
        daftarHadir[0] = new Guru();
        daftarHadir[1] = new Siswa();

        System.out.println("\n--- TAHAP 2: EKSEKUSI &
DOWNCASTING ---");
        // Melakukan perulangan untuk mengecek semua
anggota di dalam array
        for (int i = 0; i < daftarHadir.length; i++) {

            // Dynamic Method Dispatch: Tetap memanggil
versi override wujud aslinya
            daftarHadir[i].absen();

            // SENSOR KEAMANAN: Memastikan wujud asli
sebelum di-downcast
            if (daftarHadir[i] instanceof Guru) {
                // Downcasting: Membuka bungkus Induk
menjadi Guru
                Guru g = (Guru) daftarHadir[i];
                g.mengajar(); // Kemampuan spesifik
kembali pulih
            }
            else if (daftarHadir[i] instanceof Siswa) {
                // Downcasting: Membuka bungkus Induk
menjadi Siswa
                Siswa s = (Siswa) daftarHadir[i];
                s.ujian(); // Kemampuan spesifik kembali
pulih
            }

            System.out.println("-----");
        }
    }
}

```

```
}
```

Penjelasan Alur Kode

1. *Upcasting* (Baris 33-37)

Guru dan Siswa dimasukkan ke dalam satu array *AnggotaSekolah* (Induk). Di tahap ini, kemampuan khusus mereka disembunyikan dan sistem hanya mengenali mereka sebagai anggota sekolah biasa.

2. Polimorfisme (Baris 42)

Saat *Method* *absen()* dipanggil, mesin Java otomatis mengeksekusi aksi sesuai wujud asli objek di dalam memori (Guru atau Siswa), bukan versi Induknya.

3. Sensor *instanceof* (Baris 45 & 50)

Kita tidak bisa langsung memanggil *mengajar()* karena terhalang label Induk. Jadi, kita gunakan *instanceof* sebagai satpam untuk mengecek wujud aslinya (misal: "Apakah benda di laci ini benar seorang Guru?"). Ini mencegah program *error* (*Crash*).

4. *Downcasting* (Baris 47 & 52)

Jika hasil sensor benar, kita buka paksa label Induknya dan kembalikan ke tipe aslinya menggunakan sintaks (*Guru*). Setelah wujud aslinya kembali, *Method* spesifik seperti *mengajar()* atau *ujian()* akhirnya bisa dieksekusi dengan aman.

4.3 Polimorfisme pada Banyak Objek

Memanggil *Method* secara manual satu per satu (*daftar[0].absen()*, *daftar[1].absen()*) akan sangat tidak efisien jika jumlah datanya mencapai ribuan. Kekuatan sejati polimorfisme baru akan terlihat saat digabungkan dengan Perulangan (*Looping*). Misalnya, saat sistem LMS seperti *Resolva* di SMKN 1 Surabaya memproses ratusan data warga sekolah, kita bisa memanipulasi seluruh objek tersebut hanya dengan hitungan baris kode yang sangat sedikit.

Bayangkan Kepala Sekolah memberi instruksi saat apel pagi. Ia tidak mendatangi siswa dan guru satu per satu, melainkan menggunakan mikrofon (*Looping*) untuk memberikan satu perintah umum: "Laksanakan tugas!". Secara dinamis, ratusan warga sekolah akan merespons instruksi yang sama tersebut dengan aksi yang berbeda-beda sesuai perannya.

Untuk mengeksekusi kumpulan objek (*Array of Objects*) di Java, sangat disarankan menggunakan *Enhanced For-Loop* karena penulisan sintaksnya jauh lebih rapi dan kita tidak perlu mengurus indeks angka (*i = 0*).

Berikut adalah contoh penerapan kode

Tabel 4. 3 Contoh Penerapan Polomorfisme pada Banyak Objek

```
// 1. SUPERCLASS (Induk / Kerangka Dasar)
Class WargaSekolah {
    public void laksanakanTugas() {
        System.out.println("Melaksanakan tugas umum di
lingkungan sekolah.");
    }
}

// 2. SUBCLASS (Guru)
Class Guru extends WargaSekolah {
    @Override
    public void laksanakanTugas() {
        System.out.println("GURU : Berjalan ke kelas dan
membuka modul Pemrograman Java.");
    }
}

// 3. SUBCLASS (Siswa)
Class Siswa extends WargaSekolah {
    @Override
    public void laksanakanTugas() {
        System.out.println("SISWA : Duduk di bangku,
membuka laptop, dan mengerjakan sintaks PBL.");
    }
}

// 4. SUBCLASS (Staf TU)
Class StafTU extends WargaSekolah {
    @Override
    public void laksanakanTugas() {
        System.out.println("STAF TU: Menyalakan komputer
admin dan memantau server LMS Resolva.");
    }
}

// 5. PROGRAM UTAMA (Tempat Eksekusi)
public Class Main {
    public static void main(String[] args) {

        System.out.println("--- PERSIAPAN APEL PAGI ---");
        // TAHAP 1: Upcasting Massal ke dalam satu Array
        (Koleksi Objek)
        // Berbagai wujud objek ditampung ke dalam satu
        wadah berlabel 'WargaSekolah'
        WargaSekolah[] pesertaApel = {
            new Guru(),
            new Siswa(),
            new Siswa(),
            new StafTU(),

```

```

        new Guru()
    };

    System.out.println("\n---      INSTRUKSI      KEPALA
SEKOLAH: LAKSANAKAN TUGAS! ---\n");

    // TAHAP 2: Looping Polimorfik (For-Each)
    // "Ambil setiap data di dalam 'pesertaApel',
    panggil dengan nama 'individu'"
    for (WargaSekolah individu : pesertaApel) {

        // Puncak Keajaiban: HANYA 1 BARIS KODE!
        // Tidak perlu mengecek wujud asli menggunakan
        if-else atau instanceof.
        // Mesin Java otomatis menjalankan aksi
        spesifik sesuai wujud asli di memori.
        individu.laksanakanTugas();

    }

    System.out.println("\n--- APEL SELESAI ---");
}
}

```

Penjelasan Singkat Alur Eksekusi:

1. Penyatuan Data (*Upcasting*): Pada baris 33-39, kita menaruh 5 orang dengan peran berbeda-beda (Guru, Siswa, Staf) ke dalam satu barisan array `pesertaApel`. Sistem menerima mereka semua sebagai `WargaSekolah`.
2. Eksekusi Bersih (Clean Code): Perhatikan baris ke-49. Di dalam looping, kita hanya memberikan satu perintah mutlak: `individu.laksanakanTugas();`.
3. Hasil Dinamis: Saat program dijalankan, output yang tercetak tidak akan sama. Pada putaran pertama akan mencetak aksi Guru (membuka modul Java), putaran kedua aksi Siswa (mengerjakan PBL), putaran keempat aksi Staf TU (memantau server Resolva), dan seterusnya. Sistem mengeksekusinya secara cerdas tanpa perlu pengondisian if-else yang panjang.

DAFTAR PUSTAKA

- Anif, M., Samsisar, & Anggoro, D. (2024). *PEMROGRAMAN BERORIENTASI OBJEK*. DEEPUBLISH DIGITAL. ISBN: 978-623-124-971-5
- Astuti, S. N. (2019). *PEMROGRAMAN BERORIENTASI OBJEK* (T. Q. Book (ed.)). PT KUANTUM BUKU SEJAHTERA. ISBN: 978-623-7398-17-2
- Hartono, B. (2023). *PEMROGRAMAN BERORIENTASI OBJEK dengan JAVA BlueJ*. Yayasan Prima Agus Teknik. ISBN: 978-623-8120-21-5
- Ida, Samsuriah, & Faisal, M. (2024). *Pemrograman Berorientasi Objek* (B. Nasution (ed.)). PT Inovasi Pratama Internasional.
- Indahyanti, U., & Astutik, I. R. I. (2025). *Buku Ajar Pemrograman Berorientasi Objek (PBO)*. UMSIDA Press. ISBN: 978-623-464-130-1
- Nugroho, A. Y. (2025). *PEMROGRAMAN BERORIENTASI OBJEK* (W. Yuliani (ed.)). CV BRAVO PRESS INDONESIA. ISBN: 978-634-7109-52-1
- Sujjada, A., & Somantri. (2024). *Buku Ajar Pemrograman Berorientasi Objek* (C. K. Sastradipraja (ed.)). Kaizen Media Publishing. ISBN: 978-623-8317-59-2